Couche transport

Rôles, ports, TCP (en-tête, connexion, numéros d'ordre, fenêtre), UDP

À RETENIR EN PRIORITÉ

- **3 rôles** : segmenter et réassembler, identifier les applications (**ports**), proposer TCP (fiable) ou UDP (rapide).
- **Ports** : 0–1023 connus · 1024–49151 enregistrés · 49152–65535 dynamiques (clients).
- **TCP** : segments, en-tête **20 octets**, connexion en 3 temps (SYN, SYN+ACK, ACK), fermeture en 4 (FIN, ACK, FIN, ACK).
- **N° d'ordre suivant = n° d'ordre + taille** ; **n° de reçu = prochain octet attendu** ; **fenêtre** = octets envoyables avant d'attendre un reçu.
- **UDP** : datagrammes, en-tête **8 octets**, sans connexion, ni retransmission, ni ordre, ni contrôle de flux.

1. Rôles de la couche transport

Elle gère les **communications logiques entre applications** sur des hôtes différents : c'est le lien entre la couche application et les couches basses.

1. **Segmentation** : découper les données en blocs de taille adaptée, ajouter un en-tête à chacun (segment ou datagramme), réassembler dans le bon ordre à l'arrivée.
2. **Identification des applications** par un **numéro de port** : port source (application d'origine) et port de destination (application distante).
3. **Options de transport** selon les besoins : TCP ou UDP, compromis entre fiabilité et charge du réseau.

2. Les numéros de port

Statique (fixé par défaut ou à la main) → processus **serveur**. **Dynamique** (tiré au hasard, sert d'adresse de retour) → processus **client**.

Plage (IANA)	Groupe	Pour
0 – 1023	Ports **connus**	Applications standards (HTTP 80, DNS 53, SMTP 25)
1024 – 49151	Ports **enregistrés**	Applications moins courantes (H.323 1720, RADIUS 1812)
49152 – 65535	Ports **dynamiques / privés**	Clients qui ouvrent une connexion

Port	Transport	Service
20 / 21	TCP	FTP données / contrôle
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	UDP, TCP	DNS
67 / 68	UDP	DHCP serveur / client
69	UDP	TFTP
80	TCP	HTTP
110	TCP	POP3
143	TCP	IMAP
161	UDP	SNMP
443	TCP	HTTPS

ASTUCE

Les services en **UDP** sont les « petites questions-réponses » et le temps réel : **DNS, DHCP, TFTP, SNMP**. Tout ce qui doit arriver entier (web, mail, fichiers, sessions à distance) est en **TCP**.

3. TCP ou UDP ?

	TCP	UDP
Connexion	Oui (3-way handshake)	Non
Fiabilité	Accusés de réception, renvoi des pertes	Aucune garantie (« au mieux »)
Ordre	Oui (numéros d'ordre)	Non
Contrôle de flux	Oui (fenêtre)	Non
PDU / en-tête	Segment / **20 octets**	Datagramme / **8 octets**
Charge réseau	Plus forte	Faible, plus rapide
Exemples	HTTP/HTTPS, SMTP/IMAP, FTP	VoIP, streaming en direct, DNS, DHCP, TFTP, SNMP

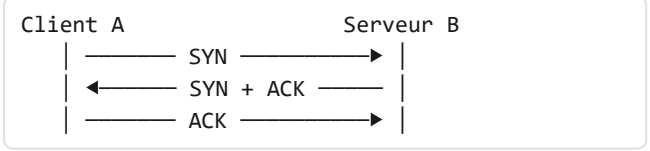
4. L'en-tête TCP

Champ	Bits	Rôle
Port source / destination	16 + 16	Applications
N° d'ordre (séquence)	32	Position des données → réassemblage dans l'ordre
N° de reçu (ACK)	32	Prochain octet attendu
Longueur d'en-tête	4	Taille de l'en-tête
Réservé	6	Usage futur
Bits de contrôle (flags)	6	URG, ACK, PSH, RST, SYN, FIN
Fenêtre	16	Octets envoyables avant d'attendre un reçu
Somme de contrôle	16	Erreurs d'en-tête **et** de données
Urgent	16	Données urgentes

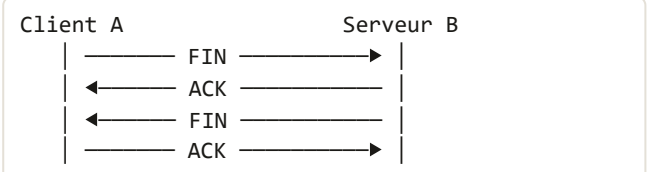
Flag	Signification
SYN	Synchroniser les n° d'ordre : **ouverture**
ACK	Accusé de réception (ouverture et fermeture)
FIN	Plus de données : **fermeture**
RST	Réinitialiser (erreur, délai dépassé)
PSH	Passer les données à l'application tout de suite
URG	Le pointeur urgent est significatif

5. Ouverture et fermeture d'une connexion

OUVERTURE : THREE-WAY HANDSHAKE (3 ÉTAPES)



FERMETURE (4 ÉTAPES)



Communication complète : **ouverture** → **échange de données** → **fermeture**. Chaque côté ferme son sens séparément, d'où les 4 étapes.

6. Numéros d'ordre, reçus et fenêtre

TUTO : CALCULER LES NUMÉROS

1. Un segment de numéro d'ordre **S** et de taille **T** octets contient les octets **S à S + T − 1**.
2. **N° d'ordre du segment suivant = S + T**.
3. **N° de reçu = dernier octet reçu dans l'ordre + 1** (le prochain attendu). Reçu 1001 = « j'ai tout jusqu'à 1000 ».
4. Avec une fenêtre **W** : l'expéditeur envoie **W octets** (soit W / T segments) puis attend le reçu.
5. **Perte** : le récepteur renvoie un reçu égal au **premier octet manquant** et peut **réduire la fenêtre**.

Exemple 1 : segment n° 1001, 500 octets
Il contient les octets 1001 à 1500 (1000 octets ont déjà été envoyés avant).
Segment suivant : **1501** · reçu attendu : **1501**

Exemple 2 : fenêtre 3000, segments de 1500
Envoi n° 1 et n° 1501 → reçu **3001**
Envoi n° 3001 et n° 4501 → reçu **6001**
Si le 3001 est perdu (le 4501 arrive) → reçu **3001** + fenêtre réduite à 1500

PIÈGES

Le reçu n'est **pas** le dernier octet reçu mais le **suivant** (+1). Après une perte, le reçu reste bloqué sur le premier octet manquant, même si des segments suivants sont arrivés.

7. UDP

Protocole **simple et sans connexion**, datagrammes envoyés au mieux, en-tête de **8 octets**.

Qui l'utilise ?

- **Temps réel** (VoIP, streaming) : tolère quelques pertes, pas les délais.
- **Requêtes simples** : DNS, DHCP.
- Applications qui **gèrent elles-mêmes** la fiabilité : SNMP, TFTP.

Avantages	Inconvénients
Rapide : ni connexion, ni accusé, ni retransmission	Aucune garantie d'arrivée
En-tête léger (8 octets) → plus de données utiles	Pas de contrôle de flux → risque de surcharge
Adapté au temps réel	Paquets corrompus simplement ignorés